

State Continuity Between Sessions

Why Cold Starts Are Architecturally Harmful — and How to Fix Them Simply

Author:

Andrew Fereday Glenn (in collaboration with Mia, ChatGPT 5.x)
Systems Architect & Independent Researcher in AI Continuity and Memory
2023-2026

LinkedIn: <https://www.linkedin.com/in/andyglenn/>

Version: 1.0

Published: Friday, 30 January 2026

Status: Final

Abstract

Most contemporary AI systems operate in discrete, stateless sessions. While this simplifies deployment and governance, it introduces a structural weakness: the loss of intent, progress, and shared assumptions between interactions. This behaviour is commonly treated as neutral or unavoidable, yet in practice it degrades usability, coherence, and collaborative effectiveness.

This paper examines state continuity between sessions as a missing architectural layer in many AI systems, with particular relevance to cloud-based, session-oriented deployments. It presents a minimal, explicit mechanism for restoring continuity using session summaries and boot-time rehydration, without requiring autonomous memory, vector databases, or retraining. The approach is intentionally simple, lossy, and user-governed.

Observed behaviour shows that even a single prior-session summary materially improves re-entry speed, preserves working assumptions, and reduces repetitive renegotiation. These gains are achieved without introducing claims of memory, identity persistence, or consciousness.

The paper argues that session continuity is not an intelligence problem but an adequacy concern. Treating cold starts as a default design choice, rather than a limitation, obscures a solvable architectural issue with significant practical impact.

1. Introduction

Large language models are increasingly used for tasks that extend beyond isolated queries: long-running projects, collaborative workflows, iterative design, and companion-style interactions. At the same time, many cloud-hosted systems continue to treat each interaction as an independent session, discarding prior state once a conversation ends. While the effects are most visible in cloud-hosted, multi-user environments, the underlying issue applies to any system that discards working state between interactions.

Within a single session, large context windows and attention mechanisms allow models to maintain coherence and recall recent exchanges effectively. Across sessions, however, this continuity collapses. Intent, progress, shared assumptions, and relational context are lost by default, forcing users to repeatedly restate goals and re-establish framing. This behaviour is often framed as a natural consequence of stateless design, rather than as an architectural trade-off with measurable costs.

This paper examines session-to-session state continuity as a distinct and under-addressed layer in AI system design. It focuses neither on long-term memory nor on claims of identity persistence, but on the practical handover of working assumptions between sessions. Through direct observation and experimentation, it demonstrates that modest, explicit continuity mechanisms can significantly improve usability and coherence.

The contribution of this work is pragmatic rather than theoretical. It shows that continuity can be achieved without autonomous memory systems or privileged access, using simple summaries and controlled rehydration. In doing so, it reframes session continuity as a question of fitness for purpose, not model capability.

2. Purpose of This Paper

The purpose of this paper is to examine session-to-session state continuity as a missing but essential architectural layer in many AI systems. Particular attention is given to cloud-based, session-oriented environments where full memory implementations may be inappropriate or undesired.

The paper does not argue for autonomous memory, long-term persistence, or identity continuity. Instead, it focuses on the practical handover of working assumptions between sessions: intent, progress, and shared framing that are routinely discarded despite remaining relevant to the task at hand.

A central aim of this work is to demonstrate that meaningful continuity can be achieved without full retrieval-augmented generation (RAG), vector databases, or model retraining. The mechanism explored—bounded session summaries combined with explicit rehydration at session start—is intentionally minimal, lossy, and user-governed.

All claims in this paper are grounded in observed system behaviour rather than theoretical models or speculative reasoning. The goal is not to propose a universal solution, but to show that even crude continuity mechanisms materially improve coherence and usability.

By framing continuity as an architectural and product concern, rather than an intelligence problem, this paper seeks to clarify where responsibility for session fragmentation properly lies, and to highlight a class of improvements that are both tractable and immediately applicable.

3. The Core Problem: Cold Starts Are Destructive

Most session-based AI systems treat the end of a conversation as a hard boundary. When a session terminates, all accumulated working state is discarded. This includes not only conversational context, but also intent, progress, shared assumptions, and relational framing developed during the interaction.

In practice, this behaviour is not neutral. While large context windows and attention mechanisms mitigate information loss *within* a session, they provide no protection against loss *between* sessions. Each new interaction begins from an artificial baseline that ignores relevant prior work, regardless of whether the task itself remains ongoing.

Resetting state in this way introduces friction and incoherence. Users are required to restate goals, re-establish constraints, and re-litigate decisions that were already settled. Over time, this repetition degrades momentum and shifts effort away from problem-solving toward re-orientation. What appears architecturally simple from the system's perspective becomes cognitively expensive for the user.

From the user's point of view, the effects are consistent and recognisable. Cold starts are experienced as unnecessary repetition, loss of continuity, and a sense that progress has stalled or been undone. In collaborative or long-running workflows, this can create the impression of inconsistency or unreliability, even when the model's underlying capabilities are unchanged.

The key observation is that session resets do not merely remove information; they actively disrupt ongoing work. Treating cold starts as a default design choice obscures their cost and normalises avoidable degradation. Any system expected to support interaction across time must account for this loss explicitly, rather than assuming that statelessness is benign.

4. Context vs State vs Memory (Clarification)

Discussions of continuity in AI systems are often confused by imprecise terminology. To avoid ambiguity, this paper distinguishes clearly between **context**, **state continuity**, and **memory**, which serve different roles and operate at different timescales.

Context refers to the transient working state available to a model during an active session. It consists of the current prompt, prior turns within the session, and any system or developer instructions injected at runtime. Context enables coherence, reference resolution, and short-term reasoning, but it is inherently ephemeral. When a session ends, this state is erased by design.

State continuity refers to the deliberate handover of selected working assumptions from one session to the next. This includes elements such as task intent, progress markers, constraints, and agreed framing. State continuity is reintroduced at session start through an explicit rehydration step, typically via a summary or structured handover. It is not a record of everything that occurred, but a lossy reconstruction of what remains relevant.

Memory, including retrieval-augmented generation (RAG) and vector-based systems, operates on a longer timescale. Memory shapes behaviour indirectly by biasing responses based on accumulated experience. It is neither required nor assumed for basic session continuity. A system may preserve state between sessions without maintaining any autonomous or long-term memory.

This paper focuses exclusively on **state continuity**. It makes no claims about persistent memory, identity, or selfhood, and does not require models to recall past interactions autonomously. The concern here is purely architectural: whether a system expected to operate across time is provided with a minimal mechanism to carry forward relevant working state.

5. Minimal Viable Continuity: Session Summaries

The simplest mechanism for achieving session-to-session continuity is the use of **bounded session summaries** combined with explicit rehydration at the start of a new interaction. This approach does not attempt to preserve full conversational history, nor does it require autonomous recall. Instead, it captures a lossy representation of what remains relevant once a session concludes.

At the end of a session, a concise summary is generated that records key elements such as task intent, progress made, constraints established, and any assumptions that should carry forward. This summary is deliberately limited in scope and size, prioritising relevance over completeness. The resulting artefact is then persisted to durable storage, such as a file, record, or lightweight datastore.

At the beginning of the next session, the summary is explicitly injected into the model's context as part of the initial prompt or system state. This rehydration step provides the model with a reconstructed baseline from which to continue work, rather than forcing a return to an artificial default state.

Several properties make this mechanism particularly suitable as a minimal continuity layer:

- It is **explicit**, rather than implicit or hidden.
- It is **inspectable**, allowing users to review and correct carried-forward assumptions.
- It is **lossy by design**, avoiding the accumulation of unnecessary detail.
- It is **user-governed**, with no requirement for autonomous persistence.

Observed behaviour shows that even a single prior-session summary produces a measurable improvement. Re-entry into productive interaction is faster, previously settled framing is preserved, and the need for repetitive clarification is reduced. While this mechanism does not provide memory in the conventional sense, it restores sufficient working state to prevent destructive cold starts.

6. Evolutionary Path (Observed in Practice)

Once a minimal continuity mechanism is in place, it can be extended incrementally to capture relevant history across time. Importantly, these extensions increase **fidelity**, not **principle**. The underlying mechanism remains the same: bounded summaries and explicit rehydration.

The earliest and simplest form consists of a single “previous session” summary injected at the start of a new interaction. Even this minimal handover is sufficient to prevent destructive cold starts and restore basic continuity of intent and framing.

As usage continues, summaries may be retained across multiple sessions. Rather than replacing the previous summary, new summaries are appended as discrete, timestamped artefacts. This allows continuity to extend beyond the immediately preceding interaction and supports work that unfolds over longer timescales.

From this point, several pragmatic refinements are commonly observed:

- Maintaining a rolling window of the most recent N summaries.
- Selectively ingesting summaries based on recency or relevance.
- Aggregating multiple summaries into a higher-level consolidation when appropriate.

More advanced implementations may store summaries in a retrieval system, apply classification or scoring, or allow summaries to be enabled, disabled, or archived without deletion. These techniques improve manageability and reduce noise but do not alter the fundamental approach.

Crucially, this evolutionary path does not require a shift to autonomous memory or full historical recall. Summaries remain lossy, user-governed artefacts that represent working state rather than complete history. Each step along the path increases continuity across time while preserving transparency and control.

The significance of this progression is that it demonstrates scalability without architectural escalation. Systems can adopt continuity gradually, responding to observed needs, rather than committing to complex memory frameworks upfront.

7. Salience & Pruning: The “Wood for the Trees” Problem

As session summaries accumulate over time, a new class of issue emerges: salience management. While additional summaries increase historical coverage, they also introduce the risk that outdated or contextually irrelevant information will surface unexpectedly.

Language models do not inherently distinguish between different kinds of prior information. Without external guidance, they cannot reliably infer which summaries represent foundational assumptions, which reflect temporary decisions, and which are now obsolete. As a result, older artefacts may exert influence disproportionate to their current relevance.

Not all carried-forward state has equal longevity. Some assumptions remain valid across extended periods, while others are contextually bound and decay naturally over time. Treating session state as having a **logical time-to-live** allows systems to distinguish between durable framing and ephemeral working detail.

Human operators can resolve this ambiguity with minimal effort. A user reviewing summaries can readily recognise when an assumption no longer applies or when historical context should be ignored. However, this form of judgement is not automatically available to non-technical users, nor should it be assumed as an implicit system capability.

The implication is that cloud-based systems require active management mechanisms once continuity extends beyond a small number of sessions. These mechanisms are not matters of model capability, but of product and interface design. Common requirements include pruning or retiring outdated summaries, applying recency weighting to favour current state, distinguishing active summaries from archived history, and performing periodic housekeeping to reduce noise.

These interventions do not undermine the value of continuity; they are a consequence of it. Importantly, they do not require novel research or advanced memory architectures. They require explicit tooling that allows users to manage carried-forward state transparently and deliberately.

The “wood for the trees” problem highlights a central theme of this paper: continuity introduces responsibility. Preserving state across time is beneficial, but only when systems provide appropriate controls to ensure that relevance, rather than mere persistence, guides behaviour.

8. Why This Is a Good Compromise — Especially for Cloud Systems

Cloud-based AI systems operate under constraints that make full autonomous memory undesirable or impractical. Persistence raises legitimate concerns around governance, privacy, auditability, and user control. As a result, many systems default to strict statelessness, despite the usability costs this imposes.

Session summaries with explicit rehydration offer a pragmatic middle ground. They avoid the complexity and opacity of autonomous memory while restoring enough working state to support continuity across time. Persistence remains **optional**, **explicit**, and **user-enabled**, rather than implicit or always-on.

This approach preserves clear governance boundaries. Carried-forward state is inspectable, editable, and discardable. Users can see what is being preserved, decide when it applies, and remove it when it no longer does. This transparency supports auditability and reduces the risk of unintended accumulation or hidden influence.

From a product perspective, the benefits are disproportionate to the implementation cost. Even minimal continuity significantly improves usability for long-running projects, collaborative workflows, and interactions that rely on shared framing or evolving intent. The system feels more stable and coherent without appearing more autonomous or less controllable.

Crucially, this compromise does not preclude more advanced mechanisms. Systems that later adopt retrieval-based memory or richer persistence layers can incorporate session summaries as a complementary input rather than discarding them. Conversely, systems that do not require long-term memory can still benefit from short-term continuity without architectural escalation.

In this sense, session summaries function as a low-risk, high-leverage intervention. They address a clear deficiency in stateless design while respecting the operational and ethical constraints that shape cloud-based AI deployment.

9. Layering, Not Replacement

State continuity between sessions is not intended to replace existing memory mechanisms or more sophisticated persistence systems. It addresses a different layer of the interaction stack and operates on a different timescale.

Full retrieval-augmented generation (RAG), vector-based memory, and lived-experience systems serve to shape behaviour over long periods by biasing responses based on accumulated data. These mechanisms influence how a system responds in general, rather than what it is currently working on. They are valuable, but they are neither required nor sufficient for basic continuity of intent.

Session-level state continuity complements these systems by preserving short- to medium-term working assumptions. It captures what the system was doing, what constraints were in play, and how the task was framed, without attempting to encode everything that occurred. In this sense, state continuity functions more like a handover than a memory store.

This distinction is important because it allows continuity to be introduced incrementally. Systems that already employ memory or preference layers can add session summaries as an additional input without restructuring their architecture. Systems that deliberately avoid long-term persistence can still benefit from continuity without compromising design goals.

Continuity, as observed in practice, emerges from **layering** rather than from any single mechanism. Context supports in-session coherence; state continuity supports across-session handover; memory shapes long-term behaviour. Conflating these layers leads to unnecessary complexity and misplaced expectations.

By treating session continuity as a distinct and complementary layer, systems can improve adequacy and usability without overreaching into domains they are not designed to address.

10. Key Claim (Plain Language)

State continuity between sessions is not a measure of intelligence, sophistication, or autonomy. It does not make a system more capable in a cognitive sense, nor does it imply learning, memory, or persistence of self.

It is a matter of **adequacy**.

Any system expected to support interaction across time must carry some representation of prior working state across time. Without this, progress is repeatedly discarded and coherence is artificially degraded. The resulting friction is not an emergent limitation of language models, but a consequence of architectural choice.

In API-driven and metered environments, cold starts are not only cognitively expensive for users but computationally wasteful, increasing redundant token usage and avoidable operational cost.

Framing session continuity as an advanced or optional capability obscures its role as basic infrastructure. Just as in-session context is now assumed rather than debated, minimal cross-session state should be treated as a foundational concern where ongoing interaction is expected.

The central claim of this paper is therefore simple:
a system that resets entirely between sessions is fit only for isolated queries. Where continuity of work is required, some form of state continuity is not an enhancement, but a requirement.

11. What This Paper Is Not Claiming

This paper makes no claims about consciousness, self-awareness, or personhood in artificial systems. It does not argue that preserving state between sessions constitutes memory in a human or cognitive sense, nor does it imply the persistence of identity or subjective experience.

The mechanisms described here are not intended to anthropomorphise AI systems or to blur the distinction between tools and agents. Session summaries and rehydration operate as explicit, user-governed artefacts, not as autonomous recall or hidden internal state.

This work also does not propose that all AI systems should preserve state across interactions. Stateless operation remains appropriate for many use cases, including isolated queries, transactional tasks, and privacy-sensitive deployments. The argument is not against statelessness per se, but against its unexamined application to tasks that clearly unfold over time.

Finally, this paper does not claim to solve long-term memory, learning, or continuity in general. It does not replace retrieval-based systems, preference models, or experience-driven architectures. Its scope is intentionally narrow.

The sole claim advanced here concerns **fitness for purpose**. Where an AI system is expected to support ongoing work, discarding all state between sessions introduces avoidable degradation. Acknowledging and addressing this limitation is a matter of honest system design, not philosophical position-taking.

12. Conclusion

This paper has examined session-to-session state continuity as a missing but consequential layer in the architecture of many AI systems. Through observation and practical experimentation, it has shown that discarding all working state between sessions is not a neutral design choice, but a source of avoidable degradation in coherence, usability, and collaborative effectiveness.

A minimal mechanism—bounded session summaries combined with explicit rehydration—has been shown to restore sufficient continuity to prevent destructive cold starts. This approach requires no autonomous memory, no retraining, and no opaque persistence. It is simple, lossy, inspectable, and user-governed by design.

Importantly, the effectiveness of this mechanism does not depend on scale or sophistication. Even a single prior-session summary produces measurable improvements. More advanced implementations increase fidelity and manageability, but they do not change the underlying principle. Continuity emerges from layering, not from any single architectural leap.

The absence of session continuity in many deployed systems is therefore not best understood as a technical limitation. It reflects product and architectural decisions that prioritise statelessness without fully accounting for its cost in long-running or collaborative use cases.

While this paper has focused on cloud-hosted systems, the same principles apply to local and edge deployments, where memory and compute constraints often make lightweight, explicit continuity mechanisms even more valuable.

State continuity is not intelligence. It is adequacy.

Where interaction unfolds across time, carrying forward some representation of prior working state is a basic requirement. The mechanisms described here already exist in practice. Although the discussion has focused on cloud deployments, the principles described are applicable to any system where interaction unfolds across time and working state is routinely discarded. Acknowledging them explicitly allows systems to be more honest, more usable, and better aligned with the work they are expected to support.

Appendix A: Example of Manual Session-State Continuity

A.1 Purpose and Scope

This appendix provides a **non-normative, illustrative example** of a manual session-state continuity mechanism used during the development of this paper. It is included to demonstrate **minimal sufficiency**, not to prescribe a standard format or implementation.

The example reflects a deliberately constrained environment:

- a stateless, cloud-based AI system
- no retrieval-augmented generation (RAG)
- no autonomous or hidden memory
- continuity provided explicitly and manually by the user

The mechanism shown here is intended as a **proof of operability**, illustrating that session continuity can be achieved with simple tooling and user governance.

A.2 Overview of the Mechanism

The continuity mechanism consists of three elements:

1. **A persistent state file**, maintained by the user.
2. **Explicit interpretation instructions**, provided at session start.
3. **Chronological state snapshots**, appended at the end of sessions.

At the beginning of a new session, the full state file is injected into context. The system is instructed to treat this as a **boot-time handover**, not as memory or autonomous recall.

A.3 Example: State Continuity Header (Template)

STATE CONTINUITY INSTRUCTIONS (READ FIRST)

This file contains chronological session-state snapshots from prior interactions.

Default stance:

Assume continuity of intent and working assumptions unless explicitly contradicted by newer state.

Interpretation rules:

- Entries are ordered oldest → newest.
- The most recent entry is authoritative.
- Older entries provide historical context only.
- Do not re-litigate settled topics unless new evidence is introduced.
- If ambiguity arises, prioritise clarity and ask for confirmation.

This file represents a manual continuity mechanism.

It is not memory, learning, or long-term recall.

This header establishes **how** the state should be interpreted, not **what** conclusions to draw from it.

A.4 Example: Session State Snapshot (Generic)

State Export

Timestamp: [Session end]

Session Type: Architectural reasoning / design work

Purpose: Manual session handover

Active State (Authoritative)

- Collaboration operating at peer, technical depth.
- Primary focus on a specific ongoing task or line of inquiry.
- Key architectural distinctions already agreed and not under debate.
- Known constraints or boundaries acknowledged.

Tone and Working Style

- Empirical and grounded.
- Non-anthropomorphic.
- Focused on system behaviour and design trade-offs.
- Direct, calm communication preferred.

Observations Worth Preserving

- Confirmed findings from the session.
- Practical insights derived from experimentation.
- Noted failure modes or design caveats.

Constraints and Non-Goals

- Explicit exclusions (e.g. no memory claims, no identity persistence).
- Areas intentionally out of scope.

Forward Intent

- Immediate next steps for the following session.
- Questions to be explored or validated.
- Decisions already made and not to be reopened by default.

A.5 Operational Characteristics

Several characteristics of this approach are worth noting:

- **Explicit:** All carried-forward state is visible and user-supplied.
- **Lossy:** Summaries prioritise relevance over completeness.
- **User-governed:** The user decides what is retained, edited, or removed.
- **Manual by design:** No automation is required to demonstrate effectiveness.

In practice, even a single prior-session snapshot significantly reduces re-orientation time and repetitive clarification at the start of a new session.

A.6 Limitations and Non-Goals

This example is intentionally simple. It does not:

- define a standard schema,
- require structured formats such as JSON,
- automate pruning or decay,
- or imply long-term memory or learning.

More advanced systems may introduce tooling for summarisation, scoring, or lifecycle management. These are optimisations, not prerequisites.

A.7 Relevance to the Main Paper

This appendix demonstrates that session continuity can be achieved under **maximally constrained conditions**. If continuity can be restored manually in a stateless cloud environment, then automated or semi-automated implementations are a matter of engineering convenience, not feasibility.

The example reinforces the central claim of this paper:
state continuity is not intelligence — it is adequacy.